

DSA — Sliding Window

Code First • Java Only • One Topic Per Page

Format

1. Sliding Window Type
2. Java Code
3. Simple Explanation

Chapter Flow

1. Fixed Size Sliding Window
2. Dynamic Size Sliding Window
3. Frequency / HashMap Sliding Window
4. Replacement / Operation Sliding Window
5. Counting — At Least K
6. Counting — At Most K
7. Longest Substring Without Repeating Characters
8. 25 LeetCode Questions
9. One-Minute Revision

1. FIXED SIZE Sliding Window

Sliding Window Type

Fixed Size → window size = K

Java Code

```
// FIXED SIZE Sliding Window

int i = 0, j = 0;
int sum = 0;

while (j < n) {
    sum += arr[j];           // add current element

    if (j - i + 1 == k) {
        System.out.println(sum);

        sum -= arr[i];       // remove previous element
        i++;
    }

    j++;
}
```

Example / Core Code

```
static double findMaxAverage(int[] arr, int k) {
    int n = arr.length;
    int i = 0, j = 0;

    double avg = Double.NEGATIVE_INFINITY;
    double cur = 0;

    while (j < n) {
        cur += arr[j];

        if (j - i >= k - 1) {
            avg = Math.max(avg, cur / k);

            cur -= arr[i];
            i++;
        }

        j++;
    }

    return avg;
}
```

Simple Explanation:

Take exactly K elements. Add the new element. When the window reaches K, calculate the answer, remove the left element, move left, and continue.

2. DYNAMIC SIZE Sliding Window

Sliding Window Type

Dynamic Size → K is not fixed. The condition decides the window size.

Java Code

```
// General Dynamic Window

int i = 0, j = 0;

while (j < n) {

    // expand
    addElement(arr[j]);

    // shrink if invalid
    while (conditionIsViolated()) {
        removeElement(arr[i]);
        i++;
    }

    // update answer
    updateAnswer();

    j++;
}
```

Example / Core Code

```
// Longest valid window – skeleton

int i = 0, j = 0, ans = 0;

while (j < n) {
    addElement(arr[j]);

    while (conditionIsViolated()) {
        removeElement(arr[i]);
        i++;
    }

    ans = Math.max(ans, j - i + 1);
    j++;
}
```

Simple Explanation:

Expand using j. If the window becomes invalid, move i until it becomes valid again. Then update the answer.

3. FREQUENCY / HASHMAP Sliding Window

Sliding Window Type

Frequency Window → validity depends on element/character counts.

Java Code

```
// Frequency Window

Map<Character, Integer> map = new HashMap<>();

while (j < s.length()) {

    // add
    map.put(s.charAt(j),
        map.getOrDefault(s.charAt(j), 0) + 1);

    // shrink
    while (conditionIsViolated()) {

        char c = s.charAt(i);

        map.put(c, map.get(c) - 1);

        if (map.get(c) == 0)
            map.remove(c);

        i++;
    }

    // update answer
    j++;
}
```

Example / Core Code

```
// Permutation / Anagram idea

int[] need = new int[26];
int[] window = new int[26];

need['a' - 'a']++;
need['b' - 'a']++;

window[s.charAt(j) - 'a']++;

window[s.charAt(i) - 'a']--;

if (Arrays.equals(need, window)) {
    // permutation found
}
```

Simple Explanation:

Store the frequency of every element inside the window. When the frequency condition fails, remove elements from the left until the window becomes valid.

4. REPLACEMENT / OPERATION Sliding Window

Sliding Window Type

Replacement → at most K changes are allowed.

Java Code

```
// Longest Repeating Character Replacement

int i = 0, j = 0;
int maxFreq = 0;
int ans = 0;

Map<Character, Integer> map = new HashMap<>();

while (j < s.length()) {

    char c = s.charAt(j);

    map.put(c, map.getOrDefault(c, 0) + 1);

    maxFreq = Math.max(maxFreq, map.get(c));

    int changes = (j - i + 1) - maxFreq;

    while (changes > k) {

        char left = s.charAt(i);

        map.put(left, map.get(left) - 1);

        i++;

        changes = (j - i + 1) - maxFreq;
    }

    ans = Math.max(ans, j - i + 1);

    j++;
}
```

Example / Core Code

```
// Example

s = "AABABBA"
k = 1

window = "AABA"
length = 4
maxFreq = 3

changes = length - maxFreq
         = 4 - 3
         = 1

1 <= k → valid
```

Simple Explanation:

Keep the most frequent character. Change all other characters. If the required changes become greater than K, shrink the window.

5. COUNTING — AT LEAST K

Sliding Window Type

At least K → once the condition is true, all extensions to the right are also valid.

Java Code

```
// Count subarrays when condition becomes true

while (conditionIsMet()) {

    ans += (n - j);

    // [i..j]
    // [i..j+1]
    // [i..j+2]
    // ...
    // [i..n-1]

    removeElement(arr[i]);
    i++;
}
```

Example / Core Code

```
// Formula

ans += (n - j);

// Number of valid right endpoints:
// j, j+1, ..., n-1

// Count = n - j
```

Simple Explanation:

If `[i..j]` already satisfies “at least K”, every longer subarray starting at `i` is also valid. So add `n - j` at once.

6. COUNTING — AT MOST K

Sliding Window Type

At most K → all valid windows ending at j can be counted together.

Java Code

```
// Count subarrays when condition is valid

int i = 0, j = 0;

while (j < n) {

    addElement(arr[j]);

    while (conditionIsViolated()) {
        removeElement(arr[i]);
        i++;
    }

    ans += (j - i + 1);

    j++;
}
```

Example / Core Code

```
// Formula

ans += (j - i + 1);

// Valid windows ending at j:
// [i..j]
// [i+1..j]
// ...
// [j..j]
```

Simple Explanation:

For fixed right pointer j , every left boundary from i to j is valid. Therefore add the current window length.

7. LONGEST SUBSTRING WITHOUT REPEATING

Sliding Window Type

Dynamic + Set / Frequency → every character in the window must be unique.

Java Code

```
// Java – HashSet

static int lengthOfLongestSubstring(String s) {

    int i = 0, j = 0, ans = 0;

    Set<Character> seen = new HashSet<>();

    while (j < s.length()) {

        if (!seen.contains(s.charAt(j))) {

            seen.add(s.charAt(j));

            ans = Math.max(ans, j - i + 1);

            j++;

        }

        else {

            seen.remove(s.charAt(i));

            i++;

        }

    }

    return ans;

}
```

Example / Core Code

```
// Java – HashMap

static int lengthOfLongestSubstring(String s) {

    int i = 0, j = 0, ans = 0;

    Map<Character, Integer> map = new HashMap<>();

    while (j < s.length()) {

        char c = s.charAt(j);

        map.put(c, map.getOrDefault(c, 0) + 1);

        while (map.get(c) > 1) {

            char left = s.charAt(i);

            map.put(left, map.get(left) - 1);

            if (map.get(left) == 0)

                map.remove(left);

            i++;

        }

        ans = Math.max(ans, j - i + 1);

        j++;

    }

    return ans;

}
```

Simple Explanation:

Add s[j]. If a character repeats, remove characters from the left until the duplicate disappears. Then update the

maximum length.

8. 25 LeetCode Questions — In Sequence

Java only. Solve in this order.

#	LC	Problem	Concept
1	643	Maximum Average Subarray I	Fixed K
2	1456	Maximum Number of Vowels in a Substring of Given Length	Fixed K
3	1343	Number of Sub-arrays of Size K and Average $\geq$ Threshold	Fixed K
4	1052	Grumpy Bookstore Owner	Fixed K + gain
5	2461	Maximum Sum of Distinct Subarrays With Length K	Fixed + frequency
6	209	Minimum Size Subarray Sum	Dynamic
7	3	Longest Substring Without Repeating Characters	Dynamic + Set
8	904	Fruit Into Baskets	At most 2 distinct
9	1004	Max Consecutive Ones III	At most K
10	1208	Get Equal Substrings Within Budget	Cost window
11	713	Subarray Product Less Than K	Dynamic
12	1493	Longest Subarray of 1's After Deleting One Element	Dynamic
13	1358	Number of Substrings Containing All Three Characters	Counting
14	930	Binary Subarrays With Sum	Counting
15	1248	Count Number of Nice Subarrays	Exactly K
16	567	Permutation in String	Frequency
17	438	Find All Anagrams in a String	Frequency
18	424	Longest Repeating Character Replacement	Replacement
19	2024	Maximize the Confusion of an Exam	Replacement
20	1423	Maximum Points You Can Obtain from Cards	Complement window
21	76	Minimum Window Substring	Frequency + minimum
22	992	Subarrays with K Different Integers	AtMost trick
23	1438	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	Deque
24	239	Sliding Window Maximum	Monotonic deque
25	862	Shortest Subarray with Sum at Least K	Prefix sum + deque

9. One-Minute Revision

FIXED SIZE

K fixed
→ add right
→ size == K
→ answer
→ remove left

DYNAMIC SIZE

→ add right
→ invalid?
→ remove left
→ valid
→ answer

FREQUENCY

→ Map / freq[]
→ add
→ check
→ shrink

REPLACEMENT

changes = windowLength - maxFreq
valid if changes <= K

AT LEAST K

ans += n - j

AT MOST K

ans += j - i + 1

Golden Rules

1. Window = continuous range.
2. j expands the window.
3. i removes from the left.
4. Fixed K → remove + add.
5. Longest → shrink until valid, then maximize.
6. Shortest → shrink while valid, then minimize.
7. Frequency → HashMap / frequency array.
8. Replacement → windowLength - maxFreq <= K.
9. Exactly K counting → often:
atMost(K) - atMost(K - 1).
10. Negative numbers:
classic sum window may fail.

CP Mindset:

Before coding, write: Window → State → Valid → Shrink → Answer.